RÉSUMÉ

Harry Bui

duchieubui511@gmail.com   250-884-0259   linkedin.com/in/harry-bui0511   github.com/hieuhust123
Victoria, BC, Canada

SUMMARY

Electrical & Computer Engineering MASc candidate specializing in RTL design and verification. Experienced across the hardware flow — from Verilog/VHDL design and class-based SystemVerilog verification, to FPGA systems and ASIC backend static timing analysis in Synopsys PrimeTime. Comfortable bridging hardware and software, with a background in embedded C/C++ and Python-driven ML research.

EDUCATION

M.A.Sc. in Electrical & Computer Engineering
Jan 2027 (Expected)

University of Victoria · Victoria, BC

Lab TA: Microprocessor-Based Systems, Applied Electronics & Electrical Machines, Electronic Devices, Linear Circuits · Coursework: Digital Design, System-on-Chip Engineering for Signal Processing, Optimization for Machine Learning

B.Sc. in Electrical Engineering
Aug 2017 – Nov 2022

Hanoi University of Science and Technology · Hanoi, Vietnam

Coursework: Microcontroller Systems, Analog & Digital Electronics, Embedded Systems

EXPERIENCE

Graduate Research Assistant — Dr. Xiaodai Dong's Lab
Feb 2024 – Present

University of Victoria

- Built and trained a hybrid CNN–RayBNN deep learning model in PyTorch and Rust for EEG-based drowsiness classification, running large-scale experiments on Compute Canada HPC clusters with Slurm-managed batch jobs.
- Implemented forward/backward propagation interfaces across a Python–Rust boundary, enabling end-to-end gradient flow between a PyTorch CNN and a Rust-based classifier.

Digital Physical Design Engineer
Oct 2023 – Feb 2024

CoAsia SEMI

- Completed a structured training program on the ASIC backend flow, covering synthesis, floorplanning, place & route, clock tree synthesis, and timing closure through lectures, design reviews, and report analysis.
- Performed static timing analysis in Synopsys PrimeTime, identifying critical paths and debugging setup/hold violations.
- Applied SDC clock and I/O delay constraints via Tcl scripts to define the design’s timing requirements for closure.

Software Engineer
May 2022 – May 2023

FPT Software

- Developed a C++ Matter Emulator on Linux and Raspberry Pi that simulated multiple Matter-compliant IoT devices (fans, thermostats, locks, lighting) on a single board, enabling automated protocol testing during product integration with LG Electronics.
- Wrote Bash automation scripts to auto-generate virtual IoT device clusters on Raspberry Pi 4, cutting device generation time by 80% and providing reproducible test environments for protocol testing.

PROJECTS

8-bit Programmable Timer IP with APB Slave Interface

- Designed an 8-bit Timer IP in Verilog across six RTL modules with up/down counting, four selectable clock inputs, write-one-to-clear status flags, and an AMBA APB slave using a three-state FSM, configurable wait states, and PSLVERR responses for reserved addresses.
- Evolved the procedural testbench into a class-based SystemVerilog environment with a driver, passive monitors, reference-model predictor, comparator, and functional-coverage collector connected by typed mailboxes; used clocking blocks for explicit APB timing and base-class inheritance for polymorphic directed/random test selection.
- Developed constrained-random stimulus with base and inline constraints alongside directed corner cases, exposing and fixing a false-overflow bug in which loading TCNT across the FF-to-00 boundary set the status flag without a real count event.
- Automated compilation, test selection, seed recording, directed/random UCDB capture, and coverage merge/report workflows with Make, Tcl, and Bash.

FPGA-Based Real-Time Image Processing System

- Engineered a real-time binary image processing pipeline in VHDL on a Nexys A7 FPGA, implementing morphological erosion, dilation, opening, and closing using a 3×3 structuring element.
- Implemented VGA synchronization and line-buffer timing logic to maintain 3×3 pixel-window alignment with display signals through the pipeline.

Hardware Accelerator Design in SystemC: Diffie-Hellman Key Exchange

- Designed a clocked hardware accelerator in SystemC to offload a compute-intensive digit-division function from the software side of a Diffie-Hellman key exchange, building a structural datapath (registers, multiplexers, adders, subtractors, multipliers) controlled by a Moore-type FSM.
- Replaced the default blocking FIFO communication between the hardware and software modules with a custom enable/done handshaking protocol, enabling cycle-level synchronization across the HW/SW boundary.

SKILLS

RTL & Design: Verilog · VHDL · FSM Design · APB Protocol · Datapath Design

Verification: SystemVerilog OOP · Self-Checking Testbenches · Constrained Random · Functional Coverage · Regression

EDA & Sim: QuestaSim · Xilinx Vivado · Synopsys PrimeTime (STA)

Programming: Python · C/C++ · Bash · Perl · Tcl

Tools: Git · Linux · Docker · HPC/Slurm